

PHASE 3: SEARCH & DISCOVERY — COMPLETE RE-BUILD

Fix ALL search issues. No shortcuts. No placeholders.

CURRENT BUGS TO FIX

1. Search is laggy/glitchy/bloated — rebuild from scratch, optimize performance
2. No autocomplete/suggestions — when user types “iron”, show “Iron Man”, “Iron Fist”, etc.
3. No specific search tabs — need Movies / Series / Anime / All tabs in search results
4. Old search code interfering — remove ALL old search code, rebuild clean

STEP 1: DELETE OLD SEARCH CODE

Remove EVERYTHING related to old search:

- Old search components
- Old search state management
- Old search API calls
- Old search CSS
- Any duplicate search logic

Run this command and screenshot output:

```
find . -type f -name "*.js" -o -name "*.jsx" -o -name "*.ts" -o -name "*.tsx" | xargs grep -l -i  
↪ "search" | sort
```

Audit each file. Delete or refactor. No leftover old code.

STEP 2: SEARCH DATA STRUCTURE

Use this EXACT mock data structure. Pre-load it. No API calls during search.

```
const searchData = [  
  { id: 1, title: "Iron Man", type: "movie", year: 2008, rating: 7.9, poster: "...", genres: ["Action",  
    ↪ "Sci-Fi"] },  
  { id: 2, title: "Iron Fist", type: "series", year: 2017, rating: 6.5, poster: "...", genres:  
    ↪ ["Action", "Fantasy"] },  
  { id: 3, title: "The Iron Giant", type: "movie", year: 1999, rating: 8.0, poster: "...", genres:  
    ↪ ["Animation", "Sci-Fi"] },  
  // ... all content  
];
```

Index for fast search:

```
// Build index once on app load  
const searchIndex = searchData.map(item => ({  
  ...item,
```

```
searchText: `${item.title} ${item.genres.join(" ")}`.toLowerCase()
    }));
```

STEP 3: SEARCH COMPONENT — CLEAN REBUILD

```
import { useState, useEffect, useRef, useCallback, useMemo } from "react";

function SearchOverlay({ isOpen, onClose }) {
  const [query, setQuery] = useState("");
  const [activeTab, setActiveTab] = useState("all"); // all | movies | series | anime
  const [results, setResults] = useState([]);
  const [suggestions, setSuggestions] = useState([]);
  const [recentSearches, setRecentSearches] = useState(() => {
    return JSON.parse(localStorage.getItem("recentSearches") || "[]");
  });
  const [isLoading, setIsLoading] = useState(false);
  const inputRef = useRef(null);
  const debounceRef = useRef(null);

  // Focus input when overlay opens
  useEffect(() => {
    if (isOpen && inputRef.current) {
      setTimeout(() => inputRef.current.focus(), 100);
    }
  }, [isOpen]);

  // Debounced search — 150ms for suggestions, 300ms for results
  useEffect(() => {
    if (!query.trim()) {
      setSuggestions([]);
      setResults([]);
      setIsLoading(false);
      return;
    }

    setIsLoading(true);
    clearTimeout(debounceRef.current);

    debounceRef.current = setTimeout(() => {
      const lowerQuery = query.toLowerCase().trim();

      // Suggestions: top 8 matches
      const suggestionMatches = searchIndex
        .filter(item => item.searchText.includes(lowerQuery))
        .slice(0, 8);
      setSuggestions(suggestionMatches);

      // Results: filtered by tab
      let filtered = searchIndex.filter(item => item.searchText.includes(lowerQuery));
```

```

    if (activeTab !== "all") {
      filtered = filtered.filter(item => item.type === activeTab);
    }
    setResults(filtered);
    setIsLoading(false);
  }, 150);

  return () => clearTimeout(debounceRef.current);
}, [query, activeTab]);

const handleSearch = useCallback((searchQuery) => {
  setQuery(searchQuery);
  // Save to recent
  setRecentSearches(prev => {
    const updated = [searchQuery, ...prev.filter(s => s !== searchQuery)].slice(0, 10);
    localStorage.setItem("recentSearches", JSON.stringify(updated));
    return updated;
  });
}, []);

const clearRecent = useCallback(() => {
  setRecentSearches([]);
  localStorage.removeItem("recentSearches");
}, []);

const highlightMatch = (text, query) => {
  if (!query) return text;
  const parts = text.split(new RegExp(`(${query})`, "gi"));
  return parts.map((part, i) =>
    part.toLowerCase() === query.toLowerCase()
      ? <mark key={i} style={{background:"E50914",color:"fff",padding:"0
        ↪ 2px",borderRadius:"2px"}}>{part}</mark>
      : part
    );
};

if (!isOpen) return null;

return (
  <div className="search-overlay" onClick={onClose}>
    <div className="search-container" onClick={e => e.stopPropagation()}>

      {/* Search Bar */}
      <div className="search-bar">
        <svg className="search-icon" width="20" height="20" viewBox="0 0 24 24" fill="none"
          ↪ stroke="currentColor" strokeWidth="2">
          <circle cx="11" cy="11" r="8"/><path d="m21 21-4.3-4.3"/>
        </svg>
        <input
          ref={inputRef}
          type="text"

```

```

        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search movies, series, anime..."
        className="search-input"
      />
      {query && (
        <button className="clear-btn" onClick={() => setQuery("")}>✕</button>
      )}
      <button className="close-btn" onClick={onClose}>Close</button>
    </div>

    { /* Suggestions Dropdown */}
    {query.trim() && suggestions.length > 0 && (
      <div className="suggestions-panel">
        {suggestions.map(item => (
          <div
            key={item.id}
            className="suggestion-item"
            onClick={() => handleSearch(item.title)}
          >
            <img src={item.poster} alt="" className="suggestion-thumb" />
            <div className="suggestion-info">
              <div className="suggestion-title">
                {highlightMatch(item.title, query)}
              </div>
              <div className="suggestion-meta">
                {item.type === "movie" ? "Movie" : item.type === "series" ? "TV Series" : "Anime"}
                ↪ {item.year}
              </div>
            </div>
          </div>
        )
        )}
      </div>
    )}

    { /* Tabs */}
    {query.trim() && (
      <div className="search-tabs">
        {["all", "movies", "series", "anime"].map(tab => {
          const count = tab === "all"
            ? results.length
            : results.filter(r => r.type === tab).length;
          return (
            <button
              key={tab}
              className={`search-tab ${activeTab === tab ? "active" : ""}`}
              onClick={() => setActiveTab(tab)}
            >
              {tab.charAt(0).toUpperCase() + tab.slice(1)} {count > 0 && `(${count})`}
            </button>
          );
        })}
      </div>
    )}

```

```

    }}
  </div>
)}

{/* Results Grid */}
{query.trim() && (
  <div className="search-results">
    {isLoading ? (
      <div className="skeleton-grid">
        {Array(8).fill(null).map((_, i) => (
          <div key={i} className="skeleton-card">
            <div className="skeleton-img"></div>
            <div className="skeleton-text"></div>
          </div>
        ))}
      </div>
    ) : results.length > 0 ? (
      <div className="results-grid">
        {results.map(item => (
          <ContentCard key={item.id} data={item} />
        ))}
      </div>
    ) : (
      <div className="empty-state">
        <div className="empty-icon"><img alt="Empty state icon" data-bbox="445 478 465 492"/></div>
        <h3>No results found</h3>
        <p>Try different keywords or check your spelling</p>
      </div>
    )}
  </div>
)}

{/* Recent Searches */}
{!query.trim() && recentSearches.length > 0 && (
  <div className="recent-searches">
    <div className="recent-header">
      <h3>Recent Searches</h3>
      <button onClick={clearRecent}>Clear</button>
    </div>
    <div className="recent-list">
      {recentSearches.map((term, i) => (
        <button key={i} className="recent-chip" onClick={() => handleSearch(term)}>
          <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
            ↪ strokeWidth="2">
            <circle cx="11" cy="11" r="8"/><path d="m21 21-4.3-4.3"/>
          </svg>
          {term}
        </button>
      ))}
    </div>
  </div>
)}

```

```

    })

    { /* Trending Searches */
    {!query.trim() && (
      <div className="trending-searches">
        <h3>Trending Now</h3>
        <div className="trending-list">
          {["The Odyssey", "House of the Dragon", "Disclosure Day", "Scary Movie", "The
            ↪ Mandalorian"].map((term, i) => (
              <button key={i} className="trending-chip" onClick={() => handleSearch(term)}>
                <span className="trend-rank">{i + 1}</span>
                {term}
              </button>
            ))}
        </div>
      </div>
    )}

    </div>
  </div>
);
}

```

STEP 4: SEARCH CSS — OPTIMIZED, NO LAG

```

/* Search Overlay */
.search-overlay {
  position: fixed;
  inset: 0;
  background: rgba(15, 15, 15, 0.97);
  backdrop-filter: blur(20px);
  z-index: 1000;
  animation: fadeIn 200ms ease;
  overflow-y: auto;
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

.search-container {
  max-width: 900px;
  margin: 0 auto;
  padding: 24px 16px 80px;
}

/* Search Bar */
.search-bar {
  display: flex;

```

```

    align-items: center;
    gap: 12px;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 12px;
    padding: 12px 16px;
    position: sticky;
    top: 16px;
    z-index: 10;
}
.search-bar:focus-within {
    border-color: rgba(255, 255, 255, 0.3);
    background: rgba(255, 255, 255, 0.08);
}
.search-icon {
    color: #888888;
    flex-shrink: 0;
}
.search-input {
    flex: 1;
    background: none;
    border: none;
    color: #FFFFFF;
    font-size: 18px;
    outline: none;
}
.search-input::placeholder {
    color: #666666;
}
.clear-btn {
    background: none;
    border: none;
    color: #888888;
    font-size: 24px;
    cursor: pointer;
    padding: 0 8px;
}
.close-btn {
    background: none;
    border: none;
    color: #FFFFFF;
    font-size: 14px;
    cursor: pointer;
    padding: 8px 12px;
}

/* Suggestions */
.suggestions-panel {
    background: rgba(30, 30, 30, 0.95);
    border-radius: 12px;
    margin-top: 8px;

```

```

    overflow: hidden;
    animation: slideDown 200ms ease;
}
@keyframes slideDown {
    from { opacity: 0; transform: translateY(-10px); }
    to { opacity: 1; transform: translateY(0); }
}
.suggestion-item {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 12px 16px;
    cursor: pointer;
    transition: background 150ms;
}
.suggestion-item:hover {
    background: rgba(255, 255, 255, 0.05);
}
.suggestion-thumb {
    width: 40px;
    height: 60px;
    border-radius: 4px;
    object-fit: cover;
    flex-shrink: 0;
}
.suggestion-title {
    color: #FFFFFF;
    font-size: 15px;
    font-weight: 500;
}
.suggestion-meta {
    color: #888888;
    font-size: 13px;
    margin-top: 2px;
}

/* Tabs */
.search-tabs {
    display: flex;
    gap: 8px;
    margin-top: 20px;
    padding-bottom: 12px;
    border-bottom: 1px solid rgba(255, 255, 255, 0.05);
    overflow-x: auto;
    scrollbar-width: none;
}
.search-tabs::-webkit-scrollbar {
    display: none;
}
.search-tab {
    padding: 8px 20px;

```

```

border-radius: 20px;
border: 1px solid rgba(255, 255, 255, 0.1);
background: transparent;
color: #888888;
font-size: 14px;
cursor: pointer;
white-space: nowrap;
transition: all 200ms;
}
.search-tab:hover {
border-color: rgba(255, 255, 255, 0.2);
color: #FFFFFF;
}
.search-tab.active {
background: #FFFFFF;
color: #000000;
border-color: #FFFFFF;
}

/* Results Grid */
.search-results {
margin-top: 20px;
}
.results-grid {
display: grid;
grid-template-columns: repeat(4, 1fr);
gap: 16px;
}
@media (max-width: 1024px) {
.results-grid { grid-template-columns: repeat(3, 1fr); }
}
@media (max-width: 640px) {
.results-grid { grid-template-columns: repeat(2, 1fr); gap: 12px; }
}

/* Skeleton Loading */
.skeleton-grid {
display: grid;
grid-template-columns: repeat(4, 1fr);
gap: 16px;
}
.skeleton-card {
aspect-ratio: 2/3;
border-radius: 8px;
overflow: hidden;
}
.skeleton-img {
width: 100%;
height: 100%;
background: linear-gradient(90deg, #1a1a1a 25%, #2a2a2a 50%, #1a1a1a 75%);
background-size: 200% 100%;

```

```

    animation: shimmer 1.5s infinite;
}
@keyframes shimmer {
  0% { background-position: 200% 0; }
  100% { background-position: -200% 0; }
}

/* Empty State */
.empty-state {
  text-align: center;
  padding: 80px 20px;
}
.empty-icon {
  font-size: 48px;
  margin-bottom: 16px;
}
.empty-state h3 {
  color: #FFFFFF;
  font-size: 20px;
  margin-bottom: 8px;
}
.empty-state p {
  color: #888888;
  font-size: 14px;
}

/* Recent & Trending */
.recent-searches, .trending-searches {
  margin-top: 32px;
}
.recent-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 12px;
}
.recent-header h3, .trending-searches h3 {
  color: #FFFFFF;
  font-size: 18px;
  font-weight: 600;
}
.recent-header button {
  background: none;
  border: none;
  color: #E50914;
  font-size: 14px;
  cursor: pointer;
}
.recent-list, .trending-list {
  display: flex;
  flex-wrap: wrap;

```

```

    gap: 10px;
  }
  .recent-chip, .trending-chip {
    display: flex;
    align-items: center;
    gap: 8px;
    padding: 10px 16px;
    border-radius: 24px;
    border: 1px solid rgba(255, 255, 255, 0.1);
    background: rgba(255, 255, 255, 0.03);
    color: #AAAAAA;
    font-size: 14px;
    cursor: pointer;
    transition: all 200ms;
  }
  .recent-chip:hover, .trending-chip:hover {
    background: rgba(255, 255, 255, 0.08);
    color: #FFFFFF;
    border-color: rgba(255, 255, 255, 0.2);
  }
  .trend-rank {
    color: #E50914;
    font-weight: 700;
    font-size: 12px;
  }
}

```

STEP 5: PERFORMANCE RULES

1. **NO setState in render loop** — causes infinite re-renders
2. **NO search on every keystroke** — 150ms debounce minimum
3. **NO re-filtering on every render** — use `useMemo` for results
4. **NO heavy computations** — pre-build search index once
5. **NO unnecessary re-renders** — memoize components with `React.memo`
6. **Virtualize long lists** — if results > 20, use windowing

```

// Memoized results
const filteredResults = useMemo(() => {
  if (!query.trim()) return [];
  let filtered = searchIndex.filter(item =>
    item.searchText.includes(query.toLowerCase().trim())
  );
  if (activeTab !== "all") {
    filtered = filtered.filter(item => item.type === activeTab);
  }
  return filtered;
}, [query, activeTab]);

```

STEP 6: INTEGRATION

- Search icon in top nav (desktop) opens overlay
 - Search icon in bottom nav (mobile) opens overlay
 - ESC key closes overlay
 - Click outside closes overlay
 - Route: `/search?q=iron` for shareable search URLs
-

VERIFICATION CHECKLIST

Run each test. Screenshot proof.

- Type “iron” — suggestions appear instantly (Iron Man, Iron Fist, etc.)
- Click suggestion — goes to that title
- Press Enter — shows full results grid
- Click “Movies” tab — only movies show
- Click “Series” tab — only series show
- Click “Anime” tab — only anime show
- Type gibberish — empty state shows
- Clear search — recent searches appear
- Click recent search — searches that term
- Mobile: search overlay full screen, keyboard opens
- Mobile: swipe down to close
- No lag when typing fast
- No glitching, no flickering
- Console: zero errors
- Lighthouse Performance 90

PROOF REQUIRED

1. Screenshot: Type “iron” with suggestions dropdown
2. Screenshot: Search results with tabs (All/Movies/Series/Anime)
3. Screenshot: Click “Movies” tab — filtered results
4. Screenshot: Empty state
5. Screenshot: Recent searches
6. Screenshot: Mobile search overlay
7. Console screenshot: zero errors

Score 95%+ = pass. Fix anything below 95%.

START NOW. DELETE OLD SEARCH FIRST. REBUILD CLEAN.